- **MSM 元素总量 ≈ `T_pad`**（所有域化多项式块的总和，上界）——**是** trace 的 pad 后版本，不是原始 trace。

F · ZK prove 输入 Trace 的数据量与计算负载

> **定位**：就一个问题给出**实测口径**的答案——ZK 证明器要消费的那份 trace 到底多大、随序列长 `s` 怎么涨、喂给 MSM / sumcheck 的负载是多少。只谈数据量与负载，不评估推理时序/延迟。 **读者**：FPGA 架构师、内存规划者——基于本文件决定 trace 占多大内存、以什么粒度传给加速器。 **依据**：本文件全部绝对数字来自 **GPT-2 124M 在 s=128/512/1024 三次 prove 的实际打点**（证明路径内按节点记录的元素数/查表层位宽，见 §2 实测方法），非估算。通用公式 §5 是把 GPT-2 实测线性/二次系数按模型维度缩放得到，供其他模型代入。

---

0. 结论先行

trace（`i64`, 8 B/元素）随 s 的一次前向中间张量总量，实测如下：

序列 s	trace 未填充 (GB i64)	域化后即 MLE 尺寸 (GB i64)	二次项(注意力 s^2)占比
128	**0.36**	0.48	12%
512	**2.11**	2.81	41%
1024	**6.03**	8.03	56%

> 注："未填充" = trace 在图上按原 shape 存的 i64 字节（FPGA 存 trace 的口径）；"域化后" = fieldize→MLE 前 pad 到 2 的幂后的元素数（真正进 MSM/sumcheck 的口径）。**pad 增量非普适**：GPT-2 三点实测约 +33%（1.329–1.331），随 `D(隐藏维)`/shape 变，见 §2.1/§4 系数说明。

关键判断：

- trace = **一个 s 长一次前向**的所有被证明节点输入/输出之和，不是逐 token 累计 → 单变量 `s` 的量级。
- 注意力二次项是 trace 的天花板：**query-key score、softmax 输出、attention-mask 三条独立的 `[Hq,s,s]` 张量**（每 block 各一条），$\propto 3 \cdot L \cdot Hq \cdot s^2$。`s` ≥ 数百后 trace 由它主导，线性项退居次要。
- 对 GPU/CPU 侧 **i64–Fr 总元素量恒定**（域化不改元素数，只 pad 到 2^n）；对 FPGA 侧，**存 trace 用 i64(8B)，参与 ZK 运算时才转 Fr(32B)**，所以 §3–§4 一律按 i64 元素数计。

---

1. Trace 是这么来的（决定体积的前提）

证明用的 trace 不是逐 token 拼接，而是用llm推理后的完整序列s，执行一次完整的prove推理，生成的记录节点保存为trace：

- 逐 token 循环（model/llm.rs 生成 token + 累积 kv-cache）不生产最终 trace。
- 最终 trace 来自最后一次以 `input_shape=[s]` 跑完整前向（model/llm.rs）：每个图节点放到一起算一遍，输出张量第一维 = 序列长 `s`。

因此 trace 大小 = **这次 s 长前向里，每个被记录节点（输入/输出中间量）之和**，是 `s` 的单变量量级，不是 ×token。

trace 在内存里的结构是 `Trace<D>`（model/trace.rs）：`steps: HashMap<NodeId, Step>`，每个 `Step` 存该节点 `node_inputs` + `node_outputs`（节点输出张量，第一维 = s）+ `proving_data`（层内辅助量：LayerNorm 均/方差、softmax shift 等）。类型 **`Element = i64`（8 B）**（lib.rs）。

---

2. 口径与实测方法

2.1 两个口径，两个用途

口径	定义	用途
未填充 i64	每个节点输出张量按原始 `shape().numel()` 计，×8 B	prove推理产生的trace是未填充的
域化后 (pad) i64	先 `pad_next_power_of_two()` 再 `numel()`，×8 B	**prove时候要先填充，fieldize→MLE→MSM 真正吃掉的数据**；GPT-2 实测约 +33%（非普适，见下）

> 域化前后**元素总数只变 padding 部分**，不改量的 $s+s^2$ 结构；pad 使每个张量放大到最近的 2 幂。**这只是 GPT-2 口径**：其 s=128/512/1024 三点实测总比 1.3291/1.3305/1.3313≈+33%（见 §4），来自其主导 `[s,768]` 线性张量在 $s=2^n$ 时恰好 pad 到 4/3；**并非普适常数**——pad 系数随 `D(隐藏维)` 和 shape 而变（见 §4 系数说明）。

2.2 实测打点（可复现）

证明路径内、在每个节点 `op.prove` 之前记录该节点计算参数（src/iop/prover.rs `record_layer_params`）：

```
[trace] n=3_layer-norm: in 98304 / out 98304 (unpad) · pad 131072 (num_vars=17) ·
tables=3 · bits=42
```

- **in/out 元素(未填充)**：`section`（该节点 trace Step）的真实输入/输出张量 `numel`
- **pad 元素 / num_vars**：`pad_next_power_of_two()` 后的元素数 / `log2`
- **tables / table_bits**：该节点 `LayerCtx` 的查表层个数 / 各表位宽之和

每节点一条；一个模型跑完累计全部节点。CSV 经 bench 导出，**输出文件名自动带 `模型.序列长.时间戳`**（多次执行不覆盖、便于对比）：

```
bench.gpt2_s128.022925.csv   ← 一次 s=128 运行的完整打点
bench.gpt2_s512.030344.csv
bench.gpt2_s1024.032736.csv
```

每份 CSV 含 **259 个节点**的参数列（GPT-2 整图）。

---

3. 实测逐算子分解（GPT-2，259 节点）

3.1 节点构成

算子	节点数	角色
einsum	73	线性/矩阵类：QKV、query-key score、attn-v、fc、proj
requant	98	量化精度=插入在每层后的重量化（每个均带查表层）
activation	12	GELU
layer-norm	25	LN1/LN2（12 块各 2）+ ln_f = 25
add	24	残差（12 块 ×2，含输出合并）
softmax	12	注意力 softmax
attention-mask	12	因果 mask
embeddings / positional	1 / 1	首尾
logits	1	末 token（输出不进 trace）

3.2 逐算子数据量（i64 未填充，单位 MB）

算子（节点数）	s=128	s=512	s=1024
einsum (73)	164.7	885.3	2374.6
attention-mask (12)	18.9	302.0	1208.0
softmax (12)	18.9	302.0	1208.0
requant (98)	77.1	308.3	616.6
activation (12)	37.7	151.0	302.0
layer-norm (25)	19.7	78.6	157.3
add (24)	18.9	75.5	151.0
positional (1)	0.8	3.1	6.3
embeddings (1)	0.8	3.1	6.3
logits (1)	~0	~0	~0

3.3 三条 s² 张量（注意力二次项的真身）

attention-mask 与 softmax 都是独立的整条 `[Hq,s,s]` trace 张量（12 个节点 = 12 块），加上 einsum 里的 query-key score 输出，**每个 block 有 3 条完整 `[Hq,s,s]`**：

```
s² 元素 = 3 (score+softmax+mask) × L 块 × Hq·s²       GPT-2: 3×12×12·s² = 432·s²
```

三者数值上完全等同（各 = `Hq·s²`），s=1024 时每条约 100.7 MB/block。

3.4 logits 特殊项

`final_proj` 只需证**末 token**（model/llm.rs，`[1, vocab]`，vocab=50257）：其输入大（s=1024 时 411.7 MB）但 **out 不进 trace**（argmax 是常数采样）。故 logits 只贡献首尾固定量、不随 `s` 放大。

---

4. Trace实测总量与公式（GPT-2）

三次实测（未填充 / 域化后，i64 元素数）：

s	Σ 未填充 (元素)	Σ 未填充 (MB)	Σ 域化后 (元素)	Σ 域化后 (MB)
128	44,673,280 ≈2^{26}	357.4	59,375,744 ≈2^{26}	475.0
512	263,627,776 ≈2^{28}	2,109.0	350,749,184 ≈2^{29}	2,806.0

s	Σ 未填充 (元素)	Σ 未填充 (MB)	Σ 域化后 (元素)	Σ 域化后 (MB)
1024	753,747,968 ≈2^{30}	6,030.0	1,003,488,256 ≈2^{30}	8,027.9

> ≈2^N = next power of two（向上取整最近 2 幂 $2^{\lceil \log_2(n) \rceil}$）；「域化后」即真正进 MSM/sumcheck 的元素个数，其幂次对应各节点 sumcheck 轮数上界（§6.2 ①）与 MSM 块数（÷2^{20}）。注意本表单位是 **MB（i64 存储）**——元素个数 × 8 B；与 §6.1 ① 表的「元素个数」口径可对照。

对三组实测点做闭合拟合，**精确成立**：

```
未填充:  T_un(s) = 293,714·s + 432·s²            元素   (i64)
域化后:  T_pad(s) = 390,145·s + 576·s²            元素   (i64)
```

以 MB 写（i64，×8B/1e6）：

```
T_un(s) ≈ 2.3497·s + 0.003456·s²   MB
T_pad(s) ≈ 3.1212·s + 0.004608·s²   MB
```

两项系数怎么来的（可直接核验）：

- **二次项 $432 \cdot s^2$**：注意力 **3 条 [Hq,s,s]/块 × L 块** = $3 \cdot L \cdot Hq \cdot s^2$。GPT-2 L=12, Hq=12 → $3 \cdot 12 \cdot 12 = 432$。它**只随 query 头数 Hq 走**，不随 kv 头数（GQA/MQA 省线性项、不省 s²）。域化对二次项本身几乎无增：s=2^n 时 [Hq,s,s] 的 $s^2 = 2^{\wedge n}$ 恰为 2 幂、该张量 pad 系数=1.0。
- **线性项 $293,714 \cdot s \approx 0.29M \cdot s$**：每 token 当量 ≈ 293,714 元素 ≈ 2.35 MB/token，即每层线性中间量约 $17 \cdot D$ 个隐藏维向量（einsum QKV/out/fc/proj、requant、activation、LN、add 线性部分叠加）：GPT-2 $17 \times 768 = 13,056$ B/token/层 × 12 层 + 首尾。
- **域化后系数（GPT-2 的 ≈×4/3，非普适）**：$T_{pad} = 390,145 \cdot s + 576 \cdot s^2$ 是对三点实测的**两项拟合**，不等于逐张量 pad 的项级分解。单张量 pad 到最近 2 幂、系数 ∈ [1,2)；GPT-2 三点实测总比 1.3291/1.3305/1.3313 ≈×4/3，主因其线性张量 $[2^n, 768]$：768=$3 \cdot 2^8$ → numel=3·2^(n+8)，次大 2 幂 2^(n+10)，系数 4/3（$2^2/3$）。**换 D 即变**：4096→~1.0、5120→~1.6、1152→~1.78。故通用模型套 ×4/3 只是量级近似，精确 T_{pad} 应用 §9 pad_next_power_of_two() 逐节点实算。

占比：s^2 主导随 s 抬升——s=128 12%、s=512 41%、s=1024 56%；s 继续翻倍则 trace 由注意力二次项统治。

---

5. 通用公式（任意模型代入）

5.1 公式阐述

把 §4 的 GPT-2 实测常数按模型维度缩放，得到可代入公式（i64 未填充口径）：

```
T_un(s) ≈ [3·L·Hq·s²] + [17·L·D·s] + [首尾 ≈ 6·D·s]      B
            (注意力二次项)   (线性/向量项)     (emb+pos+ln_f)
```

只依赖 4 个量：`L`(层)、`D`(隐藏维)、`Hq`(query 头数)、`s`。要点：

- `L·Hq·s²` 二次项（3 条 `[Hq,s,s]`/块）——**决定性变量是 Hq 不是总头数**，kv 头只砍线性项；
- `L·D·s` 线性项（≈17 个隐藏维当量/层）；
- 换域化口径**不应盲目每项 ×4/3**：pad 系数随 `D`/shape 变（§4 系数说明），GPT-2 的 ≈×4/3 仅是其 `D=768` 的特例。通用 `T_pad` 请按 `pad_next_power_of_two()` 逐模型实算（§9），或至少承认 ×4/3 是量级近似。

（例：Llama-2-7B `L=32,D=4096,Hq=32`、Qwen2-1.5B `L=28,D=1536,Hq=12`、Gemma-3-1B `L=26,D=1152,Hq=4` 代入即得各自曲线，可用 §9 脚本一键算。）

5.2 常见模型Trace评估

维度取自**仓库内 `config.json`**（`num_hidden_layers`/`hidden_size`/`num_attention_heads`），**Qwen2-1.5B 仓库内无 config，取公开标准结构参数、标注为外部来源**。注：GPT-2 以 §4 实测为准，此处通用公式自洽值仅校准校验用。

5.2.1 维度与系数（`T_un = [3·L·Hq]·s² + [D·(17L+6)]·s` 元素）：

模型	L	D	Hq	b2=3LHq	b1=D(17L+6)	维度来源
GPT-2	12	768	12	432	161,280	仓库 `model_cache/openai-community/gpt2`
Llama-2-7b	32	4096	32	3072	2,252,800	仓库 `.../Llama-2-7b-hf`
Llama-2-13b	40	5120	40	4800	3,512,320	仓库 `.../Llama-2-13b-hf`
Llama-3.2-1B	16	2048	32	1536	569,344	仓库 `.../LLM-Research/Llama-3.2-1B`
Gemma-3-1B	26	1152	4	312	516,096	仓库 `.../google/gemma-3-1b-it`
Qwen2-1.5B	28	1536	12	1008	740,352	**外部标准**（仓库无 config）

口径：以下六档并给出（**512 / 1024 低档 + 4k/8k/16k/32k 四档**）。4k-32k 为**满注意力图上界**——当前图 materialize 完整 3 条 `[Hq,s,s]`（§3.3），s² 项随序列长四四方方放大；真实运行还受模型**架构约束**兜底（`max_position_embeddings`、滑动注意力），见"可达性"。低档 s≤1024 的 GPT-2 实测另有 §7.2 可对（此处为通用公式自洽值）。**trace 数据量（`T_un`/`T_pad` 六档）见 §4**；此处只给耗时。

跨模型 trace 数据量外推（把 GPT-2 实测按 §5 通用公式缩放到其他模型，六档 + ‡ 越界标注；维度见 §7.4 上表）：

5.2.2 trace 数据量

（`T_un` i64 存储口径 / `T_pad` 域化后，进 MSM/sumcheck；‡ = 超该模型 `max_position_embeddings`，仅为数学上界）：`T_pad` 按各模型**实际 pad 系数分项计算**——二次项 `[Hq,s,s]` × `p(Hq)`、线性项 `[·,D]` × `p(D)`，系数 `p(n)=2^⌈log₂n⌉/n`（§4 系数说明；如 p(D):4096→1.0、5120→1.6、1152→1.78）。**GPT-2 直接锚 §4 实测多项式** `390145·s+576·s²`，不用通用公式。值均为量级参考，精确值按 §9 逐张量 `pad_next_power_of_two()` 实算。

模型	s=512 T_un / T_pad	s=1024 T_un / T_pad	s=4k T_un / T_pad	s=8k T_un / T_pad	s=16k T_un / T_pad	s=32k T_un / T_pad
GPT-2	1.57 / 2.81 GB	4.95 / 8.03 GB	63.27 / 90.09 GB ‡	0.24 / 0.33 TB ‡	0.95 / 1.29 TB ‡	3.75 / 5.05 TB ‡
Llama-2-7b	15.67 / 15.67 GB	44.22 / 44.22 GB	486.14 / 486.14 GB	1.80 / 1.80 TB ‡	6.89 / 6.89 TB ‡	26.98 / 26.98 TB ‡
Llama-2-13b	24.45 / 39.12 GB	69.04 / 110.46 GB	759.34 / 1.21 TB	2.81 / 4.49 TB ‡	10.77 / 17.23 TB ‡	42.15 / 67.44 TB ‡
Llama-3.2-1B	5.55 / 5.55 GB	17.55 / 17.55 GB	224.81 / 224.81 GB	861.95 / 861.95 GB	3.37 / 3.37 TB	13.34 / 13.34 TB
Gemma-3-1B	2.77 / 4.41 GB	6.85 / 10.13 GB	58.79 / 71.94 GB	201.33 / 227.63 GB	737.66 / 790.27 GB	2.82 / 2.92 TB
Qwen2-1.5B	5.15 / 6.86 GB	14.52 / 19.36 GB	159.55 / 212.74 GB	589.69 / 786.25 GB	2.26 / 3.02 TB	8.85 / 11.80 TB

‡（越界档）：GPT-2 连 4k 都越界（max=1024）、Llama-2 系恰好止步 4k（max=4096），这两个模型 s=4k 及以上数字只是数学上界、无实际意义（见 §7.4 "可达性"）。**非 ‡ 档 = 一条实际可达的评估曲线**（含低档 512/1024）。

6. MSM 块数 与 查表层实例（trace 的直接消费侧）

6.1 MSM（批量承诺）负载

批量承诺对每个多项式按 `min(ceil(2^num_vars / 2^20), 128)` 切块、每块一次 Pippenger MSM。域化后总元素 = `T_pad`

- **MSM 元素总量 ≈ `T_pad`**（所有域化多项式块的总和，上界）——**是** trace 的 pad 后版本，不是原始 trace。
- 因此：

```
MSM 元素总量 = T_pad(s)                                  元素
MSM 块次数  ≈ Σ_节点 ceil(pad_elems / 2^20)              上界（每多项式另有 ≤128 块 cap）
```

实测（域化后总元素 / 块次数上界）：

s	T_pad 元素	T_pad MB	MSM 块次数(上界)
128	59,375,744	475	61
512	350,749,184	2,806	337
1024	1,003,488,256	8,028	958

> MSM 块次 ∝ trace 域化元素量（$s+s^2$）；s 翻倍时 s^2 项把块次拉高 ~4×。每块 = 2^{20} 元素 = 8 MB i64 / 32 MB Fr，是 FPGA 双缓冲乒乓的建议灌入粒度。**本条只对 GPT-2 可达档成立**——它三档 s≤1024 均未触 2^{27} cap；一旦某条 `[Hq,s,s]` 多项式 ≥2^{27} 元素即被 128 块/poly 封顶、块次不再随 s^2 涨（见 6.1 ③）。

以下为跨模型 MSM 负载核算（承接 D 文档附录 A 的 chunk 语义：执行图 = `ProverSplit` + `c×ChunkProver` + `Final`，每个 `ChunkProver` 跑一段完整 `prove_chunk`）。关键结论：chunk 数 `c` 只决定 `batch_commit` 调用次数（=c），不改变 Pippenger MSM 总次数。

① 域化元素总量 T_pad 与 MSM 块次数

域化元素总量 T_pad（进批量承诺的元素总个数，单位：元素个数）：

模型	s=512	s=1024	s=4k	s=8k	s=16k	s=32k
GPT-2	350.7M≈2^{29}	1003.5M≈2^{30}	11.26G≈2^{34}‡	41.85G≈2^{36}‡	161.0G≈2^{38}‡	631.3G≈2^{40}‡
Llama-2-7b	1.96G≈2^{31}	5.53G≈2^{33}	60.77G≈2^{36}	224.6G≈2^{38}‡	861.5G≈2^{40}‡	3.37T≈2^{42}‡

模型	s=512	s=1024	s=4k	s=8k	s=16k	s=32k
Llama-2-13b	4.89G≈2^{33}	13.81G≈2^{34}	151.9G≈2^{38}	561.4G≈2^{40}‡	2.15T≈2^{41}‡	8.43T≈2^{43}‡
Llama-3.2-1B	694.2M≈2^{30}	2.19G≈2^{32}	28.10G≈2^{35}	107.7G≈2^{37}	421.6G≈2^{39}	1.67T≈2^{41}
Gemma-3-1B	551.6M≈2^{30}	1.27G≈2^{31}	8.99G≈2^{34}	28.45G≈2^{35}	98.78G≈2^{37}	365.1G≈2^{39}
Qwen2-1.5B	857.7M≈2^{30}	2.42G≈2^{32}	26.59G≈2^{35}	98.28G≈2^{37}	377.0G≈2^{39}	1.48T≈2^{41}

> 单位：**元素个数**（i64，×8 B 为字节；×32 B 为 Fr）。`≈2^N` = **next power of two = 向上取整到最近的 2 幂** `2^⌈log₂(T_pad)⌉`（如 GPT-2 s=128 的 `59,375,744 ≈ 2²⁶`，见上实测表）。这一幂次直接对应 sumcheck 每实例轮数量级（见 6.2）与 MSM 块数（÷2^{20}，见下）。**GPT-2** 锚 §4 实测多项式 `390145·s+576·s²`（精确）；**其余按分项 pad 系数**：二次项 `[Hq,s,s]`×`p(Hq)`、线性项 `[·,D]`×`p(D)`，`p(n)=2^⌈log₂n⌉/n`（§4 系数说明）——故 D/Hq 为 2 幂者 pad 系数=1.0（不 pad），非 2 幂者 >1（如 5120→1.6、1152→1.78）。精确 `T_pad` 应逐张量 `pad_next_power_of_two()`（§9）。值均为量级参考。‡ = 超该模型 `max_position_embeddings`、仅为数学上界。

按 2^{20} 整块切后的 MSM 块次数——`ceil(域化元素总量 T_pad ÷ 2²⁰)`（即「总量是 2 的多少次方」，每块一次 Pippenger MSM）：

模型	s=512	s=1024	s=4k	s=8k	s=16k	s=32k
GPT-2	335	958	10,741 ‡	39,913 ‡	153,553 ‡	602,017 ‡
Llama-2-7b	1,868	5,272	57,952	214,208 ‡	821,632 ‡	3,216,128 ‡
Llama-2-13b	4,664	13,168	144,832	535,424 ‡	2,053,888 ‡	8,039,936 ‡
Llama-3.2-1B	662	2,092	26,800	102,752	402,112	1,590,656
Gemma-3-1B	526	1,208	8,576	27,136	94,208	348,160
Qwen2-1.5B	818	2,308	25,360	93,728	359,488	1,407,104

> ✅ **这是基准口径**：`c = ceil(T_pad / 2²⁰)` 就是「把域化元素总量按 2^{20} 一块切出几块」的原始次数，随 s→大以 s^2 主导 ×~4/档。实际 `batch_commit` 内部按多项式逐块切（见下 ③），并有 `MAX_CHUNKS=128` 的实现上限——**该上限是当前代码常数、非协议硬限制**：若某多项式元素真超 `128×2²⁰`，正确实现仍需算全（扩块/分块即可），故**块次数概念上就是 ceil 值上界、不应被 cap 削减**。GPT-2 三档 335/958 与上实测 337/958 对齐（差为逐节点 ceil padding）。

② **batch_commit 调用次数（= chunk 数 c）**

- **恒 = c，与模型、与 s、与 T_pad 无关**：每个 ChunkProver 各调 1 次，把该 chunk 的**全部已域化多项式一次性**交给 PCS::batch_commit（context.rs batch_commit：一次 batched multiexp 返回按 2^{20} 切块的每块承诺）。
- 全图 c=1 → batch_commit 1 次；c=3 → 3 次；c=8 → 8 次。**增大 c 只把 Pippenger 总次数摊进更多调用**（SequentialExecutor 是逻辑切分、非 CPU 并行）。

③ Pippenger MSM 的按多项式逐块切分（实现口径）

概念上块次数就是 ① 的 ceil(T_pad/2^20)；batch_commit 的实际实现**按多项式逐块**切：每个多项式 min(ceil(2^num_vars/2^20), 128) 块、每块一次 Pippenger MSM（context.rs num_chunks_for_polynomial：DEFAULT_CHUNK_SIZE_THRESHOLD=20、MAX_CHUNKS=128）。⚠ **MAX_CHUNKS=128 是当前代码常数、非协议硬限制**：单多项式元素真超 128×2^20 时，正确实现仍需算全（把 128 上限提高/分块即可），故**① 的 ceil 上界始终成立、不被 cap 削减**——cap 只是当前实现的调度粒度约束，这里记录它以便对照实现。**每块数据恒定 2^{20} 元素 = 8 MB i64 / 32 MB Fr（g1_power 预置，见上），一次 batch_commit 喂该 chunk 的块数。**

④ MSM 每次调用的数据量（A/batch_commit）

调用	次数	每次数据量(域化后元素)	随 s 表现
A/MSM（batch_commit）	**c 次**（内部 Pippenger 总次数见上 ③）	**每块恒定 2^{20} 元素 = 8 MB i64 / 32 MB Fr**（g1_power 预置）；一次调用喂 该 chunk 块数 块	每块数据不变；块数 = ① 的 ceil、随 s^2 涨

> **FPGA 读数**：MSM 的"次"是 **2^{20} 元素整块**、常量级灌入粒度（双缓冲乒乓的天然灌入单位），比 ceil(T_pad/2^20)（①）决定的块数更刚性。MAX_CHUNKS=128 只是当前实现的调度粒度、非协议上限——超大二次项该算全还得算全，别当"工作量天花板"看。

6.2 查表层（sumcheck 实例）负载

查表层（LogUp lookup）节点数与查表层数——GPT-2 实测：

zkML 里非线性算子（RMSNorm、SiLU、softmax、反量化等）不是直接算了，而是**查表**：把函数值预先打进一张 LogUp 表，运行时按位宽查。**图里带查表支持的算子**叫"查表层节点"，**一个节点可能带多张表**。下表数清 GPT-2 图里"靠查表实现的算力"有多大：

算子（带查表的）	图里节点数	每节点带几张表	单表位宽 d_L（=$\log_2$表元素数）	表总数
layer-norm（RMSNorm）	25	3	14	75
softmax	12	3	16	36

算子（带查表的）	图里节点数	每节点带几张表	单表位宽 d_L（=log₂表元素数）	表总数
activation（SiLU）	12	3	≈15	36
requant（反量化）	98	3–4	15	318
logits	1	1	16	1
合计	**148**	—	—	**466**

> **表总数怎么算**：`表总数 = 节点数 × 每节点表数`（如 25×3=75；requant 每节点 3–4 张、混合后实测 318）。**这 466 张表面向下 ① 的实例数 V**：每张表 LogUp 要单独跑一次 sumcheck 折叠，加 148 个带表节点各自的"链接收口"，一并构成 GPT-2 V 里的**查表层-fold ≈2262**（见下 V 表，GPT-2 只实测这一组）。`d_L`=单表位宽=`log2(查表层元素数)`，只管**表多大、不随 s 变**；它决定该表折叠的轮数（量② 查表层行）。

end2end 含义：总成本 = `Σ_实例(轮数 × 每轮数据量)`，随 `s+s²` 涨（score 的 s² 实例主导）；实例数不随 s、由图拓扑定。下面把三个量逐个给公式与逐模型·逐 s 数值。

① sumcheck：可证明节点数 V × 每实例轮数 × 每轮数据量

sumcheck 对每个要证明的算子节点跑一遍协议，把节点输出的张量当一张多元多项式取值表。它的负载拆成三**个量相乘**：**实例数 V × 每实例轮数 × 每轮数据量**。

量① 实例数 V——"跑多少次协议"（不随 s，只由图节点拓扑定）：

> 节点每产生一张要 prove 的张量就算一个实例。**V ≠ 图节点数**：查表类节点内部展开成多个 fold、各自单独验，所以 V 远大于图节点数（GPT-2：259 节点 → 2467 实例）。V 决定 FPGA 能并行铺多少个 sumcheck，总数不随 s 变、只随 chunk `c` 在单次调用里分摊。

模型	V（可证明实例总数）	每 chunk c=1 / c=3 / c=8
GPT-2	**2467**（实测）	2467 / 823 / 309
Llama-2-7b	288（结构外推）	288 / 96 / 36
Llama-2-13b	360（结构外推）	360 / 120 / 45
Llama-3.2-1B	144（结构外推）	144 / 48 / 18
Gemma-3-1B	182（结构外推）	182 / 61 / 23
Qwen2-1.5B	252（结构外推）	252 / 84 / 32

> 注：**只 GPT-2 为仓库实测**（259 节点 / 466 查表层，实例数 = 2262 查表层-fold + 205 einsum/embedding/add/mask 主实例 ≈ 2467）；其余模型标"结构外推"。

量② 每实例轮数 = num_vars——"每个实例跑几轮"：

> sumcheck **每轮消去 1 个变量（1 个 bit 坐标）**。一张张量有 $\texttt{num_vars} = \log_2(\text{元素数})$ 个坐标，就要跑 num_vars 轮。⚠ **是变量数、不是维度数**：score $[Hq,s,s]$ 是 3 维，但元素 = $Hq \cdot s^2$，故 $\texttt{num_vars} = \lceil \log_2(Hq \cdot s^2) \rceil$——如 GPT-2 s=1024：$\lceil \log_2(12 \cdot 1024 \cdot 1024) \rceil = 24$ → **24 轮**。

各类张量的轮数公式（轮数 = 该张量 num_vars）：

张量	形状	num_vars（轮数）
score / softmax / mask（注意力）	$[Hq,s,s]$	$\lceil \log_2(Hq \cdot s^2) \rceil$（随 s^2 涨）
线性 matmul（QKV/fc/proj 等）	$[\cdot,s]$	$\lceil \log_2(\cdot s) \rceil \approx 20$ 上下
查表层（LogUp 每表）	2^d_L	d_L + 1（d_L≈14–16，实测上表）

s^2 主导实例（score/softmax/mask $[Hq,s,s]$）逐模型·逐档的 num_vars（= 轮数）：

模型 (Hq)	s=512	s=1024	s=4k	s=8k	s=16k	s=32k
GPT-2 (12) / Qwen2-1.5B (12)	22	24	28	30	32	34
Llama-2-7b / Llama-3.2-1B (32)	23	25	29	31	33	35
Llama-2-13b (40)	24	26	30	32	34	36
Gemma-3-1B (4)	20	22	26	28	30	32

> **两种变化**：① **随 s**：s^2 主导 → $\texttt{num_vars} = 2 \cdot \log_2 s + \log_2 Hq$，**每翻一档 +2 轮**（表内 1024→4k 是 ×4 跳档故 +4）。② **跨模型**：同一 s 只差 $\lceil \log_2(Hq) \rceil$ ≤2 轮（Hq 从 4 到 40），即**轮数几乎与模型无关**——真正拉开成本的是每轮张量 $Hq \cdot s^2$（×32）与层数 L（×~2–3）。 GPT-2 s=1024 全部实例（score/线性/查表层混合）轮数分布：$[(10,1),(20,185),(21,12),(22,24),(24,36),(26,1)]$，加权均值 ≈ 20.8 轮/实例；s=128 ≈ 17.4 轮。

量③ 每轮数据量 = 该张量 2^num_vars 次求值——"每轮要算几次"：

> 张量域化到下一个 2 的幂后的元素数 = 2^num_vars，每轮都要取一遍——**正是 6.1① 里该张量的域化后元素量**。score 类每轮取 $Hq \cdot s^2$、随 s^2 涨；线性/查表层每轮取更小的张量（$2^n \approx d_L$ 档）、随 s 线性、量级小。

比如单个 score 实例的总求值量 = 轮数 × 每轮数据量 = $\texttt{num_vars} \times Hq \cdot s^2 = \lceil \log_2(Hq \cdot s^2) \rceil \times Hq \cdot s^2$（逐模型·逐档，元素数）：

模型 (Hq)	s=512	s=1024	s=4k	s=8k	s=16k	s=32k
GPT-2 / Qwen (12)	0.07G	0.30G	5.64G	24.2G	103 G	438 G
Llama-7b / 3.2 (32)	0.19G	0.84G	15.6G	66.6G	283 G	1203 G
Llama-2-13b (40)	0.25G	1.09G	20.1G	85.9G	365 G	1546 G

模型 (Hq)	s=512	s=1024	s=4k	s=8k	s=16k	s=32k
Gemma-3-1B (4)	0.02G	0.09G	1.74G	7.52G	32 G	137 G

> **怎么读**：这是**单个** score 实例的累积求值量（每轮读一遍 $Hq \cdot s^2$、读 num_vars 轮）。
>
> - **C 段总量** = Σ_实例(轮数 × 每轮张量) = **s^2 类（≈3·L 个/模型）× 上表单实例量 + 线性/查表层实例**。
> - **随 s**：单实例 $\propto num_vars \cdot Hq \cdot s^2$，每翻一档 ≈×4 ×（轮数 +2 的缓涨），总量随 s^2 涨（对照 §7.4 耗时 ×~2.1/档，是幂律摊薄后）。
> - **跨模型**：单实例差只来自 Hq——13b 是 Gemma ~11 倍（40/4）、7b 是 GPT-2 ~2.7 倍（32/12）、GPT-2/Qwen 同 Hq=12 持平；总成本再叠层数 L。
> - **对 T_pad 的倍率（C 段为何占 84.6% 耗时）**：T_pad 里每个元素只被 MSM 承诺 1 遍；同一个 s^2 张量在 sumcheck 里却**逐轮读 num_vars 遍** → s^2 部分 ≈ T_pad 的 num_vars（20–36）倍，线性/查表层再按各自张量与轮数叠乘 ⇒ **sumcheck 总元素量 ≫ T_pad ≥ T_un**（GPT-2 s=1024 平均 ~20.8 轮 ≈ 把同口径张量读 20 遍，再乘 V≈2467）。

> **诚实边界**：表值是 s^2 家族**单实例**的精确逐模型数值（$\lceil \log_2(Hq \cdot s^2) \rceil \cdot Hq \cdot s^2$，与 6.1 口径同源）——**轮数/单实例量及其随 s、跨模型的规律每模型成立；全模型总成本 = 单实例 × V + 线性/查表层**。GPT-2 实测 V=2467 可得绝对量；**其余模型 V 仅结构下界**（每层矩阵主实例 7–9 + 查表类=0，$d_L \approx 30$；真实 LLM 仍有 RMSNorm/SiLU/softmax 查表层类节点、实际 V 显著更高），精确绝对总量 GPT-2 外均未定，需逐模型 record_layer_params 打点。

② **sumcheck 每次调用的数据量（C/sumcheck + D/合并）**

调用	次数	每次数据量(域化后元素)	随 s 表现
C/sumcheck（prove_claims）	**V 实例**（无关 c）	每实例每轮数据量 = 该节点 2^num_vars（上 ① 量③；score 的 [Hq,s,s] 每轮取 $Hq \cdot s^2$ 主导）	次数不变，单实例数据随节点张量 s 涨
D/合并+KZG	**c 次**（每 chunk 各 1）	每次 ≈ 该 chunk 全部 claim 数，随 chunk 内节点数 ≈ V/c	随 s 近似线性（边界节点数量级），×~1.6/档

> **FPGA 读数**：C 的"次"是**单实例一次折叠**、次数由图拓扑 V 定；D 的"次"是**每 chunk 一次合并+KZG 打开**（=c 次）、数据量 ≈ chunk claims 数。单实例数据随节点张量 s 涨，s^2 类实例是每轮大数据量来源。放大规模就只有换算法（滑动窗口/分块注意力，§7.4）或堆面积分摊两路。

---

7. 四阶段耗时（实测分解，GPT-2）

口径与归属：代码里没有标为 A/B/C/D 的四列，而是多处相对独立打点的聚合键。下表按**被测函数的真实覆盖范围**归并，能直接对上的键标"直接实测"，没有独立打点的 B 段由 `prove_full` 余量推算（量级极小）。

7.1 三段实测键 → A/B/C/D 归属

实测键	代码打点处	覆盖	四段归属
`chunk_split`	prover_graph.rs `ProverSplit`	full_trace 切 chunk	A（可忽略，≈1ms）
`witness_commitment`	lookup/context.rs `witness_commit`	trace→MLE→批量承诺 MSM	**A**（直接实测）
`prove_claims`	prover.rs，输入/输出边界 claims→`prove_tables`	C 节点 sumcheck + 查表层实例	**C**（直接实测，含 D 的合并部分）
`prove_commitment_opening`	prover.rs `commit_prover.prove`	合并 sumcheck / RLC / KZG 打开	**D**（直接实测）
`prove_full`	prover.rs `chunked_prove_local` 外层	整段端到端	合计

B 段（通道挑战播种 claims）在 `prove_full`≤2% 余量内，无独立打点，下表单列仅为完整四段。

7.2 耗时与占比（s=128 / 512 / 1024）

段	s=128	s=512	s=1024	占比(s=1024)	翻倍增长
A 域化+批量承诺	1.7 s	6.7 s	16.8 s	2.1%	×2.5–3.9
B 播种 claims	~1 s	~2 s	~3 s	<0.5%	—
C 节点 sumcheck	130.1 s	322.2 s	681.5 s	**84.6%**	×2.1–2.5
D 合并+KZG 打开	25.0 s	41.1 s	71.1 s	8.8%	×1.6–1.7
端到端 prove_full	160.9 s	382.8 s	805.4 s	100%	×2.1–2.4

7.3 结论（直接由实测）

- **C 段（节点级 sumcheck）是绝对瓶颈，占端到端的 81–85%**，随 s 翻倍近似 ×2.1–2.5——超线性但未到 s²，因节点 sumcheck 的实张量轮数（见 §6 MSM 块/查表层）比 MSM 块数涨得慢。优化 sumcheck 折叠/并行增益最大。
- **D 段（合并 sumcheck+KZG 打开）随 s 只 ×1.6–1.7/翻倍**，占比从 15.5%(s128) 掉到 8.8%(s1024)：它对 trace 线性随 s（边界节点数量级），不是 s² 主导。
- **A 段（MSM 批量承诺）随 s 涨最快（×2.5–3.9）**但占比小（1–2%）：它直接随 §4 `T_pad` 的 s² 项走，`s` 继续翻倍时占比会抬升、渐成第二瓶颈——与 §4 轨迹一致。
- **总时长 ≈ trace 的量级**：s 翻倍 trace 增 ~2.2× 时端到端增 ~2.24×（128→512 ×2.38、512→1024 ×2.10），证明时长与 trace 数据量同源同涨。

7.4 外推到常见模型（按 §5 通用公式）

四阶段耗时（端到端用 GPT-2 实测幂律 `a·T_pad^b`=235.87·GB^0.561 秒外推；‡ 档耗时同为超限示意。A/C/D 占比恒定 2.1 / 84.6 / 8.8%，见 §7.2）：

模型	s=512 end2end	s=1024 end2end	s=4k end2end	s=8k end2end	s=16k end2end	s=32k end2end
GPT-2	0.12 h (421 s)	0.21 h (759 s)	0.82 h	1.71 h	3.64 h	7.83 h
Llama-2-7b	0.31 h	0.55 h	2.1 h	4.4 h	9.3 h	20.1 h
Llama-2-13b	0.51 h	0.92 h	3.5 h	7.3 h	15.6 h	33.5 h
Llama-3.2-1B	0.17 h	0.33 h	1.4 h	2.9 h	6.3 h	13.5 h
Gemma-3-1B	0.15 h	0.24 h	0.7 h	1.4 h	2.8 h	5.8 h
Qwen2-1.5B	0.19 h	0.35 h	1.3 h	2.8 h	5.9 h	12.6 h

> 单档 C 段（节点 sumcheck）≈ 0.846×end2end：如 Llama-2-7b s=8k 的 C ≈ 3.7 h、Qwen2-1.5B s=32k 的 C ≈ 10.7 h。耗时档位与 trace 档位一一对应（‡ 档同为超限示意）。

可达性（能否原生到该档，由各模型 `config.json` 的 `max_position_embeddings` 判定）：

模型	max_position_embeddings	仅到	注意力架构
GPT-2	**1024**	1k	全 attention

模型	max_position_embeddings	仅到	注意力架构
Llama-2-7b / 13b	**4096**	4k	全 attention
Llama-3.2-1B	131072（rope 缩放, 原生 8192）	任意	全 attention
Gemma-3-1B	32768	32k	**sliding_window=512**（局部注意力）
Qwen2-1.5B	32768（外部标准）	32k	全 attention（GQA）

读数与长序列结论：

- **每翻一档，trace ×~4、耗时 ×~2.1**（s^2 项主导 T_pad、幂律 b=0.561 摊薄耗时）。从 s=4k 到 32k：trace ×16、耗时 ×4.6，与表内 Llama-2-7b（486 GB→26.98 TB、2.1 h→20.1 h）吻合。
- **Hq 决定 s^2 天花板**：Llama-2 系（Hq=32/40）二次项系数大 → 一过 4k 就 TB 级；Gemma-3-1B 因 Hq=4，本应增长最缓——但真正的差异是它**用滑动窗口**（见下）。**D·L 决定线性项**：模型越大单位长度 trace 越高；GQA（多个模型 nk<Hq，见 config）只省线性项、不省 s^2。
- **满 attention 图在 16k/32k 必然失败于内存与时延**：即使最小可达的 Qwen2-1.5B，s=32k 也要 11.8 TB trace、~11 h 的 C 段 sumcheck。s^2 项（3 条 `[Hq,s,s]`）是唯一膨胀源。
- **破除 s^2 是长序列唯一出路**：Gemma-3 的 `sliding_window=512` 把 score/softmax/mask 从 `[Hq,s,s]` 截为 `[Hq,s,512]`（s^2→线性 s·512），这正是它 32k 档 trace 只有 2.92 TB、远小于同档 Qwen 11.8 TB 的原因。**16k/32k 全 attention 无解；必须落在局部/分块注意力（Gemma 式）上。**
- FPGA 若需 ≥16k，规划口径是**滑动窗口化后的 trace（线性）**，非满 attention 上界：把 s^2 的 `[Hq,s,s]` 换成 `[Hq,s,W]`（W=窗宽），按 §8 流式 chunk 逐窗口增量灌给 MSM/sumcheck。
- **外推耗时仅量级参考**：幂律在 s=512 对 GPT-2 回代有 ±10% 偏差；C 段占比随并行/折叠策略可变。FPGA 以"T_pad GB"为内存真口径、耗时留一档余量。

> 一键重算：`python3 tools/extrapolate_models.py`（§9 脚本），改 `REPO/EXT` 字典加模型即得逐 s 表。

8. 可复现（脚本与命名规则）

- **打点**：`record_layer_params`（src/iop/prover.rs）在 `op.prove` 前逐节点记录元素数/num_vars/查表层；trace 总量另在 `chunked_prove_local`、`SequentialExecutor`、`instantiate_witness_ctx_for_chunk`（批量承诺前）三处打印 `[tracer]`。

- **CSV 命名**：`bench.{model}_s{max_context}.{HHMMSS}.csv`，自动含模型/序列长/时间戳，多次执行互不覆盖、便于对比。
- **对账**：`zkml/tools/dump_bench_facts.py`（看逐算子汇总）+ `zkml/tools/verify_trace_from_bench.py`（总量 + 本文件 §4 公式核对）。
- **长序列/调用次数**：§7.4 用 `zkml/tools/extrapolate_models.py`（trace + 四阶段耗时）、调用次数核算用 `zkml/tools/extrapolate_msm_sumcheck.py`（sumcheck 实例数 + **未含 128 块 cap 的 MSM 块次数上界**，按 chunk c=1/3/8 均摊）。两脚本均以 GPT-2 实测为锚、其余模型 §5 通用公式外推并标"示意"。⚠ **脚本的 MSM 块次数是按 `ceil(T_pad/2^20)` 的全局上界、未考虑 §6.1 ③ 的 128 块/多项式 cap**——在 `s≥4k` 二次项饱和区会**高估** Pippenger 次数（可到 ~2 数量级）；精确次数须按 §6.1 ③ 逐多项式 `min(2^(num_vars−20),128)` 重算。

```
python3 tools/dump_bench_facts.py                          # 打印 CSV 逐算子汇总
python3 tools/verify_trace_from_bench.py bench.gpt2_s*.csv   # 总量 + §4 公式核对
```
